

#JONSECOPS

AWS CLOUD PROJECT

-S3 DATA EXFILTRATION FORENSICS-

Table of Contents

EXECUTIVE SUMMARY: 3

PHASE 1: ENVIROMENT SET UP 4

STEP 1: CREATING S3 BUCKET 5

STEP 2: CREATING VPC CONFIGURATION AND FLOW LOG CREATION 6

STEP 3: CREATE CLOUD TRAIL 7

PHASE 2: DATA EXFILTRATION SIMULATION ATTACK IN CLOUDSHELL 9

COMMAND BREAKDOWN: 9

PHASE 3: EC2 INSTANCE ATTACKER MACHINE LAUNCH 10

PHASE 3: FORENSIC LOG ANALYSIS 14

PHASE 4: RECOMMENDATIONS 17

APPENDIX: PICERL INCIDENT RESPONSE LIFECYCLE MATRIX 18

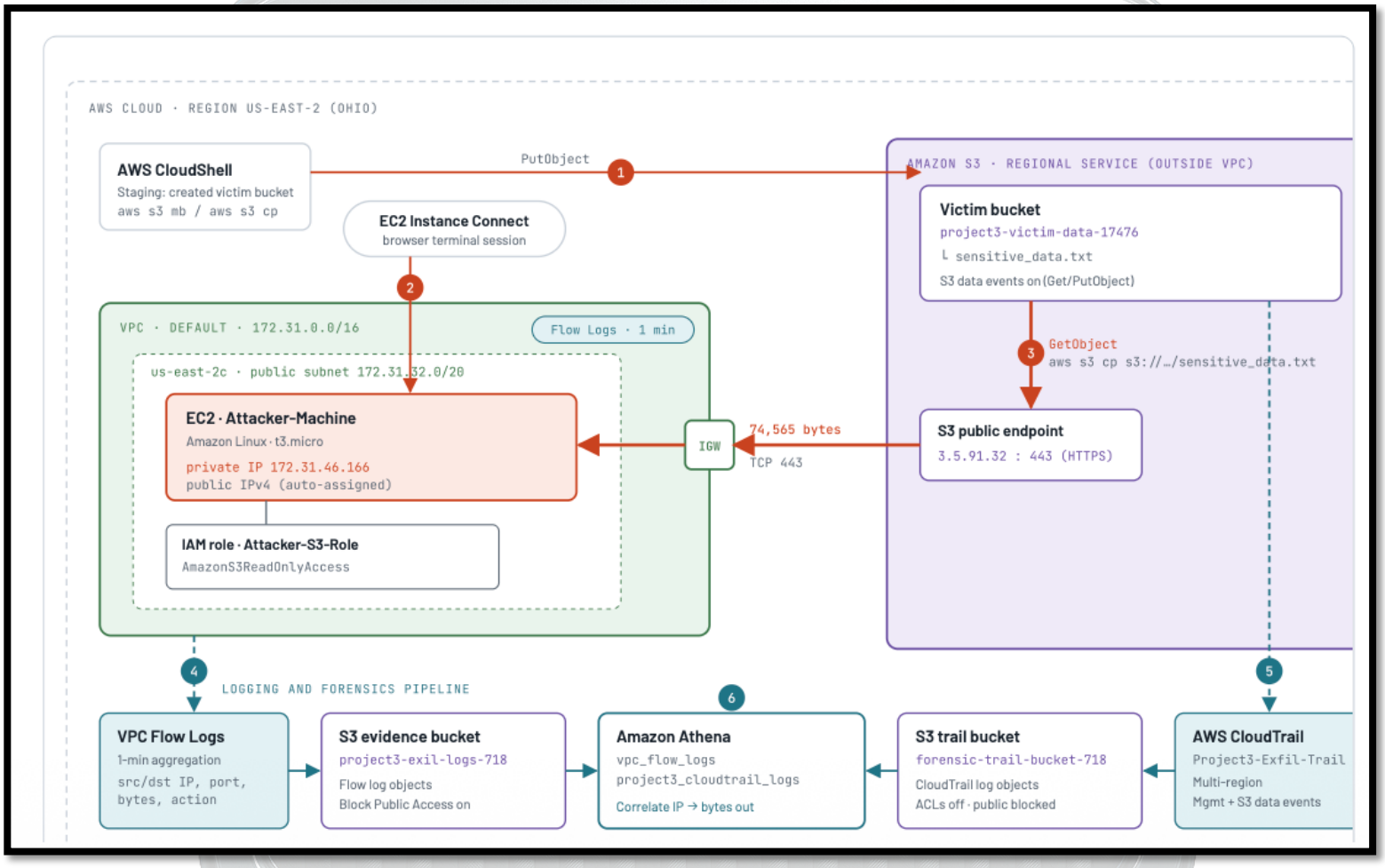
7339 86D1 JSO JonSecOps

EXECUTIVE SUMMARY:

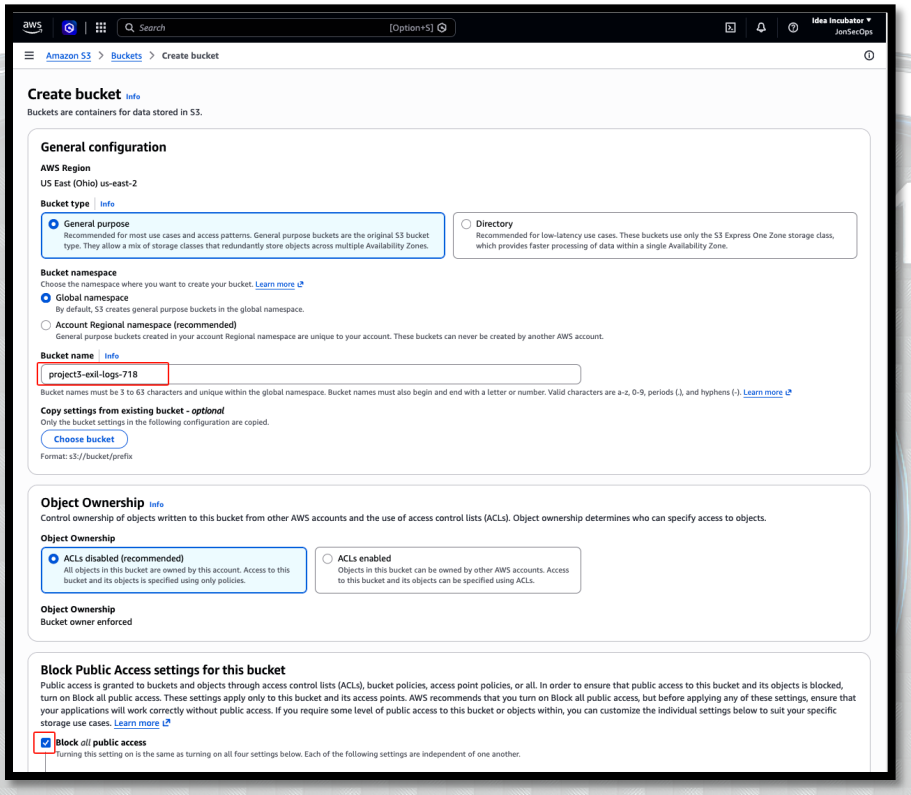
This report details a cloud forensics investigation into an S3 data exfiltration event. The environment was configured with AWS **CloudTrail** and **VPC Flow Logs** routed to a centralized Amazon S3 evidence bucket. A simulated threat actor provisioned an Amazon **EC2** instance with a modified **IAM** role and executed a command line data exfiltration attack against a sensitive S3 bucket. Utilizing **Amazon Athena**, it successfully correlated the control-plane API event (**GetObject**) with the underlying network transport layer. This correlation confirmed the exact volume of data exfiltrated (**74,565 bytes**) and identified the specific egress path, demonstrating the critical need for multi-layered cloud logging.

JSO
JonSecOps

PHASE 1: ENVIROMENT SET UP



STEP 1: CREATING S3 BUCKET



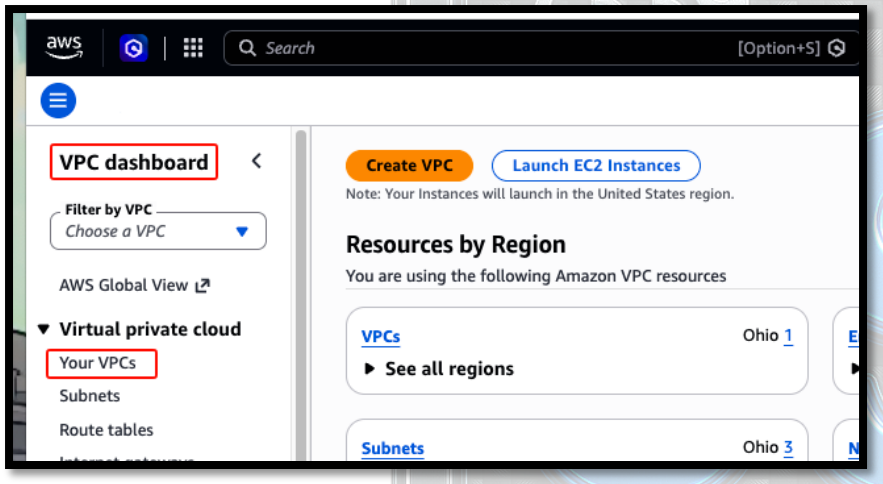
To ensure secure and reliable storage for multi-region AWS CloudTrail audit logs, a dedicated Amazon S3 evidence bucket was provisioned prior to enabling control-plane telemetry.

- **Bucket Naming & Region:** A globally unique bucket name (forensic-trail-bucket-718) was designated within the active AWS region to ensure organized evidence retention.
- **Object Ownership:** Configured with **ACLs disabled (recommended)**, shifting permission management entirely to AWS Identity and Access Management (IAM) and bucket policies in accordance with modern security standards.
- **Block Public Access: Block all public access** was strictly enabled to prevent any unauthorized external exposure of sensitive management-plane telemetry and log artifacts.

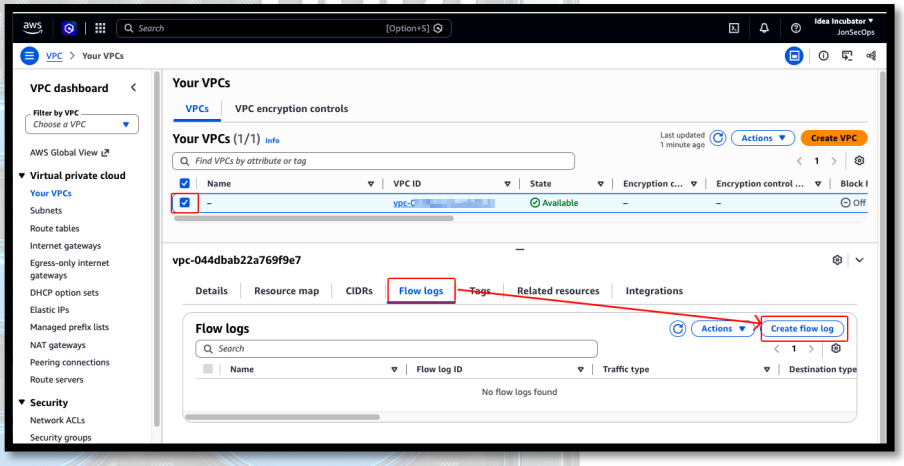
JonSecOps

STEP 2: CREATING VPC CONFIGURATION AND FLOW LOG CREATION

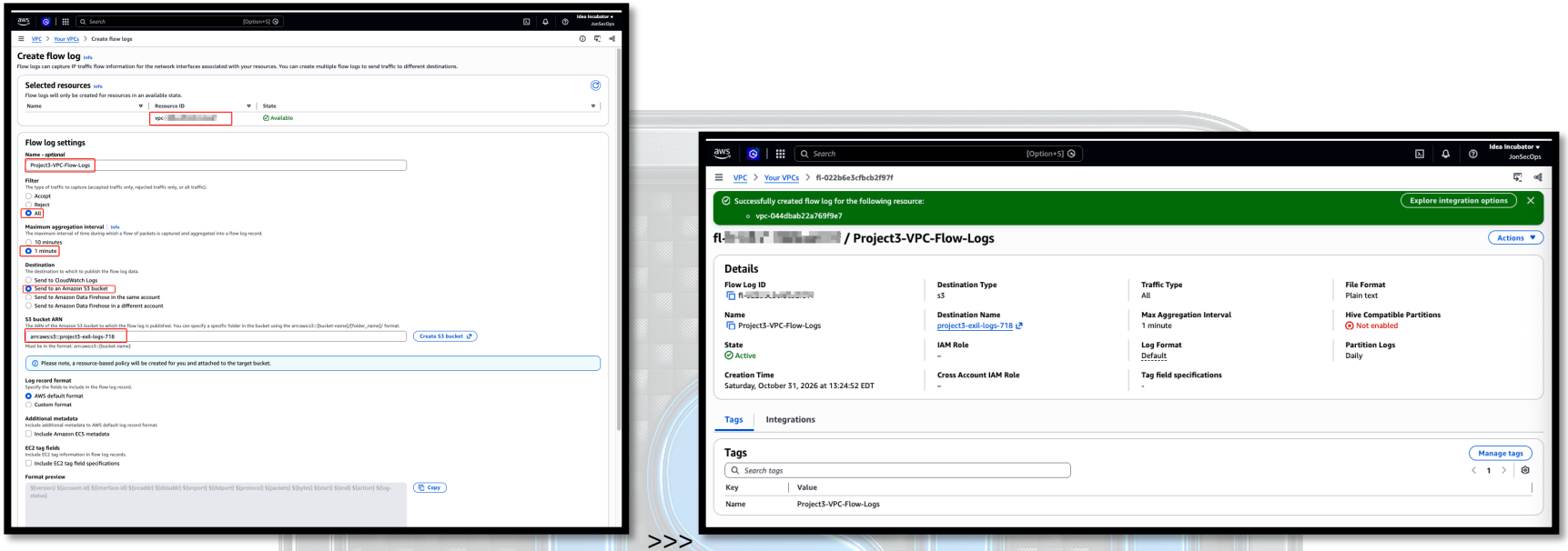
To capture network level transport data, **VPC Flow Logs** were enabled on the target **Virtual Private Cloud (VPC)**. The flow logs were configured with a 1 minute maximum aggregation interval to ensure rapid delivery and highly granular tracking of network traffic. The destination was routed directly to the centralized Amazon **S3** evidence bucket (**project3-exil-logs-718**) for secure retention and subsequent forensic querying.



>>>



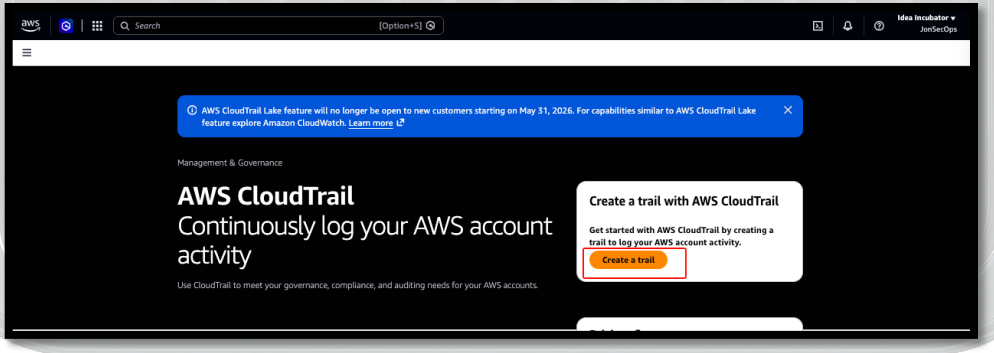
>>>

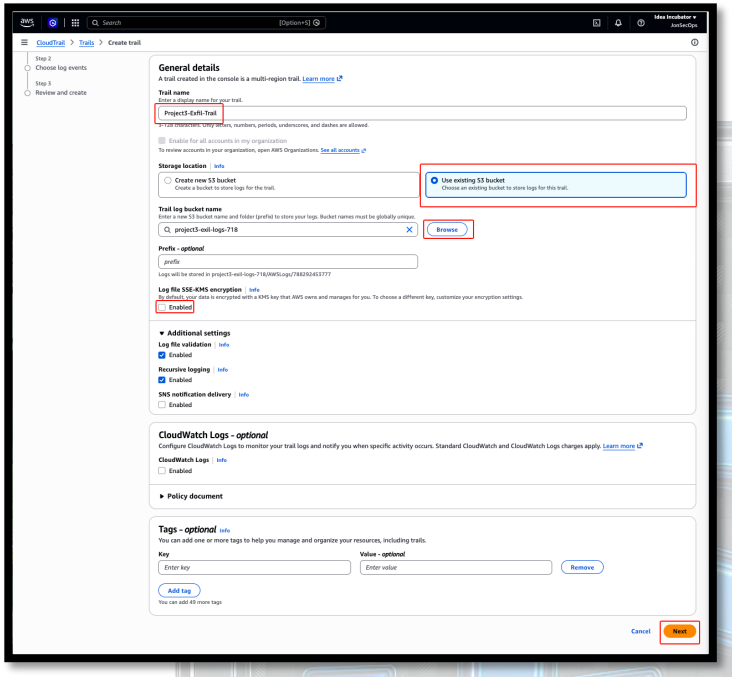


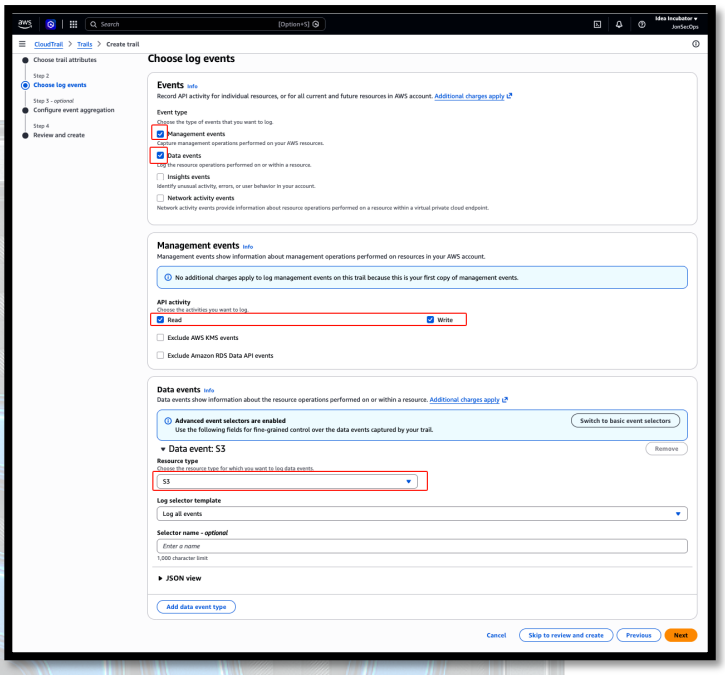
>>>

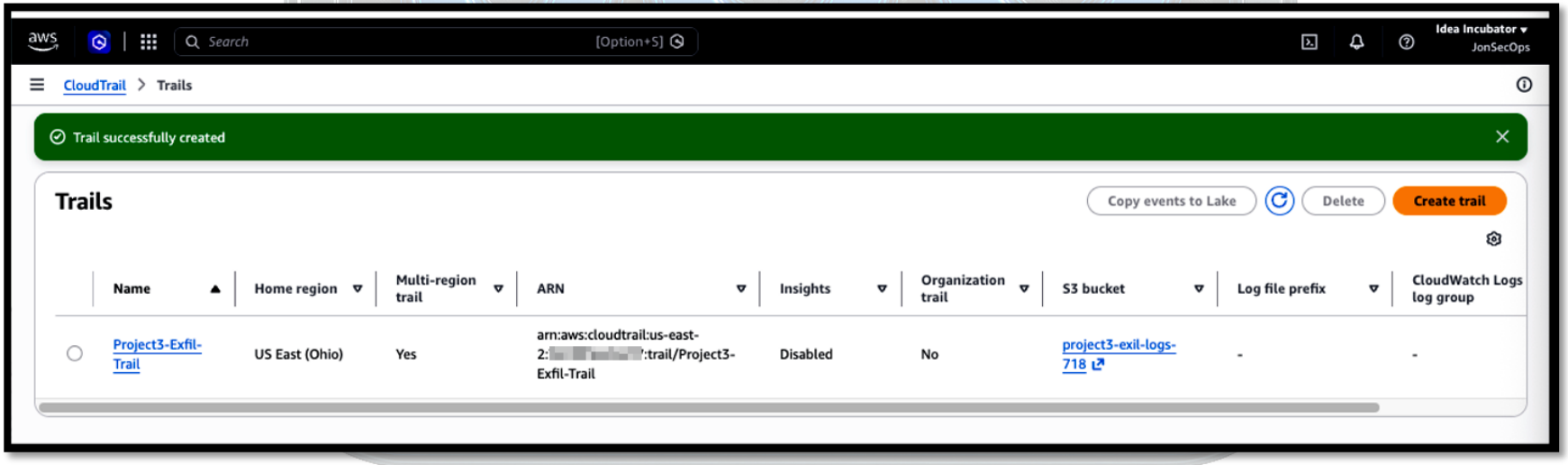
STEP 3: CREATE CLOUD TRAIL

AWS **CloudTrail** was configured to capture both management-plane and data-plane events across the environment. A multi-region trail (**Project3-Exfil-Trail**) was deployed, specifically enabling **S3** Data Events to monitor object-level operations, such as **GetObject** and **PutObject**. The resulting logs were routed to the S3 evidence bucket to provide a comprehensive and immutable audit trail of all API activity.



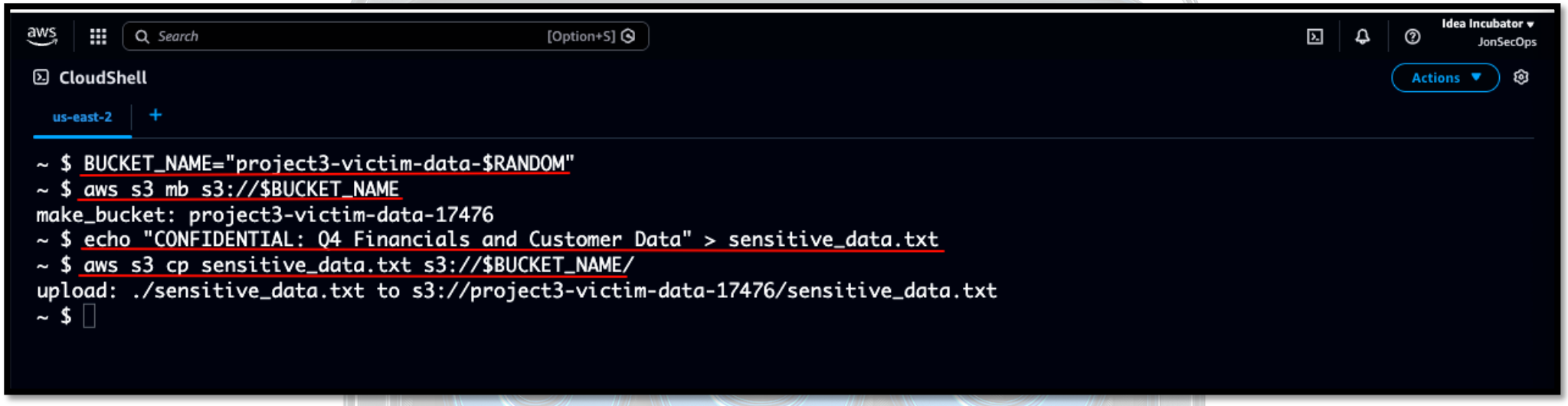

CloudTrail > Trails > Create trail
General details
Trail name
Project3-Exfil-Trail
Storage location
Use existing S3 bucket
Trail log bucket name
project3-exil-logs-718
Browse
Prefix - optional
Log file SSE-KMS encryption
Enabled
Additional settings
Log file validation
Recursive logging
SNS notification delivery
CloudWatch Logs - optional
Policy document
Tags - optional
Add tag
Cancel
Next


CloudTrail > Trails > Create trail
Choose trail attributes
Choose log events
Configure event aggregation
Review and create
Choose log events
Events
Record API activity for individual resources, or for all current and future resources in AWS account.
Event type
Management events
Data events
Insights events
Network activity events
Management events
No additional charges apply to log management events on this trail because this is your first copy of management events.
API activity
Read
Write
Exclude AWS KMS events
Exclude Amazon RDS Data API events
Data events
Data events show information about the resource operations performed on or within a resource.
Advanced event selectors are enabled
Switch to basic event selectors
Data event: S3
Resource type
S3
Log selector template
Log all events
Selector name - optional
JSON view
Add data event type
Cancel
Skip to review and create
Previous
Next


Search
[Option+S]
Idea Incubator
JonSecOps
CloudTrail > Trails
Trail successfully created
Trails
Copy events to Lake
Delete
Create trail
Name
Home region
Multi-region trail
ARN
Insights
Organization trail
S3 bucket
Log file prefix
CloudWatch Logs log group
Project3-Exfil-Trail
US East (Ohio)
Yes
arn:aws:cloudtrail:us-east-2:[redacted]:trail/Project3-Exfil-Trail
Disabled
No
project3-exil-logs-718
-
-

PHASE 2: DATA EXFILTRATION SIMULATION ATTACK IN CLOUDSHELL

We will now simulate the data exfiltration attack. This requires staging sensitive data in a new bucket, and then simulating an attacker exfiltrating that data through the VPC network to trigger the flow logs.



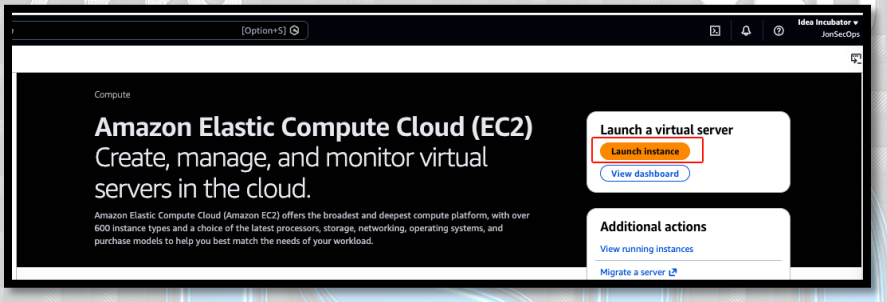
COMMAND BREAKDOWN:

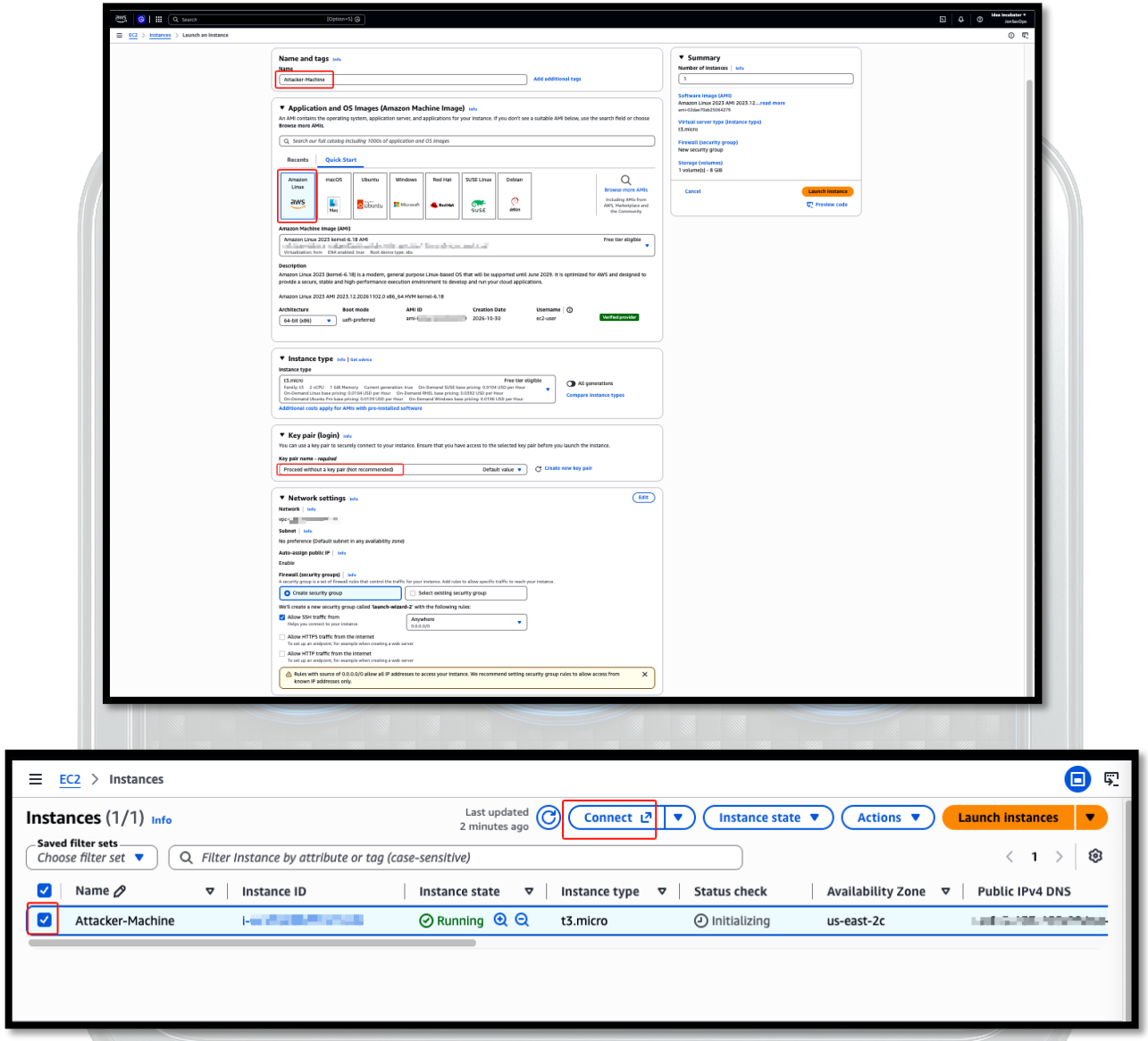
- **BUCKET_NAME="project3-victim-data-$RANDOM"** generates a unique bucket name by appending a random number to a string, preventing naming conflicts.
- **aws s3 mb s3://$BUCKET_NAME** instructs the AWS CLI to "make bucket" (mb), officially provisioning the new S3 storage resource in your account.
- **echo "..." > sensitive_data.txt** creates a local text file within your CloudShell environment containing a simulated string of confidential financial information.
- **aws s3 cp sensitive_data.txt s3://$BUCKET_NAME/** copies the local text file into your newly created S3 bucket, which directly triggers the PutObject data events that CloudTrail is now actively monitoring.

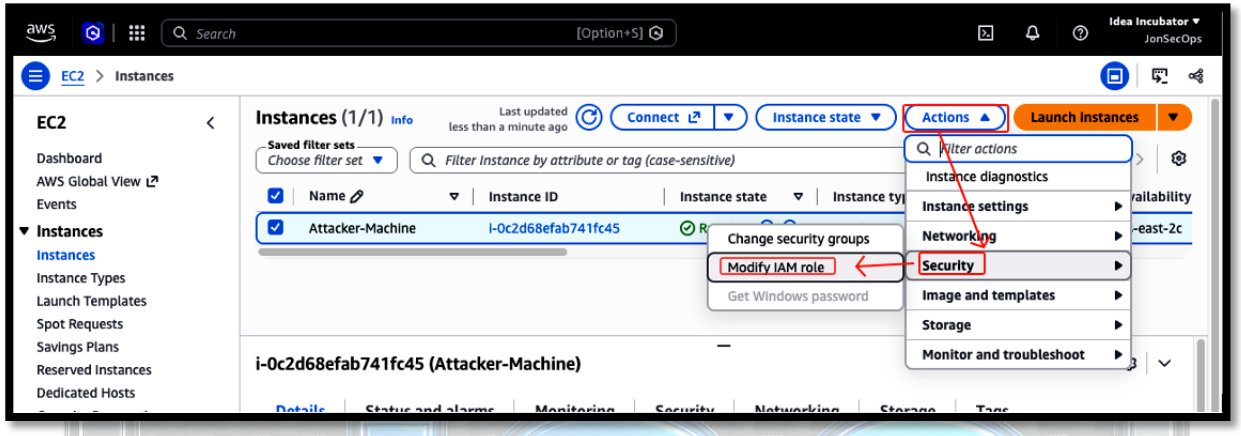
JonSecOps

PHASE 3: EC2 INSTANCE ATTACKER MACHINE LAUNCH

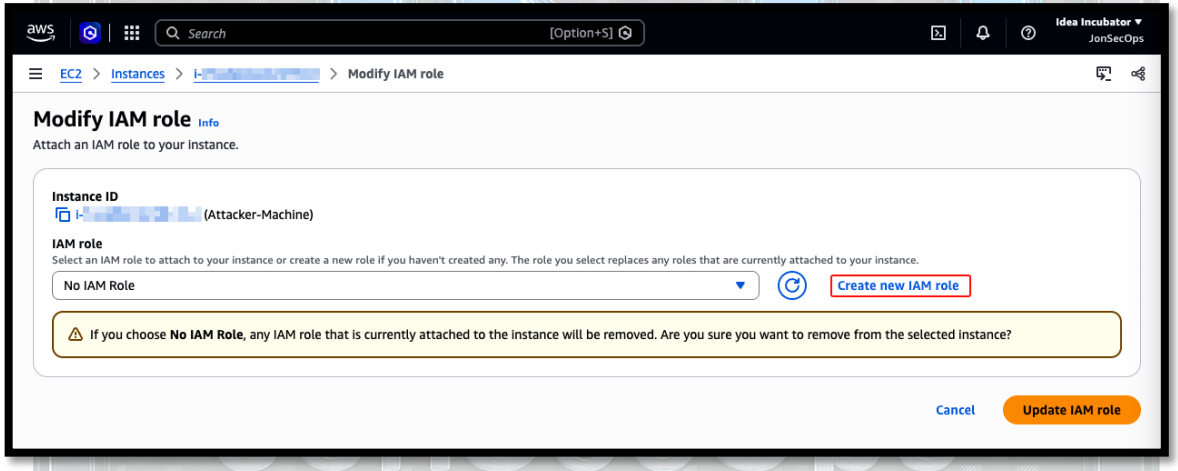
A simulated adversary environment was established by launching an Amazon Linux **EC2 instance** designated as the Attacker-Machine. To facilitate the attack, the instance's **IAM** profile was modified to attach a new role containing **AmazonS3ReadOnlyAccess**, granting the necessary privileges to read objects from the victim S3 bucket. A terminal session was then established using **EC2 Instance Connect** to execute the unauthorized data exfiltration command.

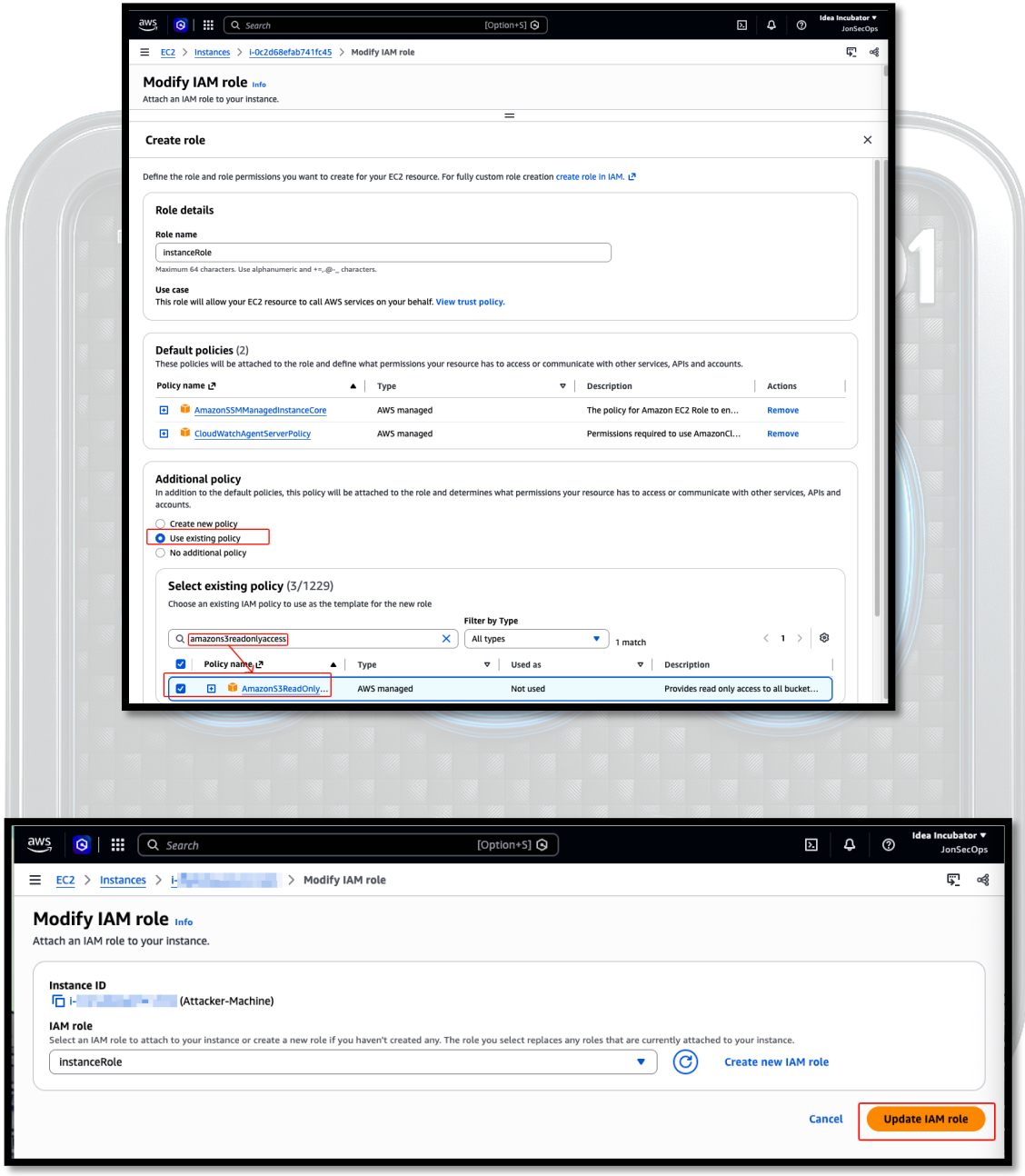



Name and tags
Attacker-Machine
Application and OS Images (Amazon Machine Image)
Summary
Instance type
Key pair (login)
Proceed without a key pair (Not recommended)
Network settings
Launch instance
EC2 > Instances
Instances (1/1) Info
Last updated 2 minutes ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Filter Instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
Attacker-Machine
Running
t3.micro
Initializing
us-east-2c



The Attacker-Machine IAM role needed to be updated.




Search
[Option+S]
Idea Incubator
JonSecOps
EC2 > Instances > i-0c2d68efab741fc45 > Modify IAM role
Modify IAM role Info
Attach an IAM role to your instance.
Create role
Define the role and role permissions you want to create for your EC2 resource. For fully custom role creation create role in IAM.
Role details
Role name
instanceRole
Maximum 64 characters. Use alphanumeric and +=,.@-_ characters.
Use case
This role will allow your EC2 resource to call AWS services on your behalf. View trust policy.
Default policies (2)
These policies will be attached to the role and define what permissions your resource has to access or communicate with other services, APIs and accounts.
Policy name
Type
Description
Actions
AmazonSSMManagedInstanceCore
AWS managed
The policy for Amazon EC2 Role to en...
Remove
CloudWatchAgentServerPolicy
AWS managed
Permissions required to use AmazonCl...
Remove
Additional policy
In addition to the default policies, this policy will be attached to the role and determines what permissions your resource has to access or communicate with other services, APIs and accounts.
Create new policy
Use existing policy
No additional policy
Select existing policy (3/1229)
Choose an existing IAM policy to use as the template for the new role
Filter by Type
amazons3readonlyaccess
All types
1 match
Policy name
Type
Used as
Description
AmazonS3ReadOnly...
AWS managed
Not used
Provides read only access to all bucket...
Search
[Option+S]
Idea Incubator
JonSecOps
EC2 > Instances > i- > Modify IAM role
Modify IAM role Info
Attach an IAM role to your instance.
Instance ID
i- (Attacker-Machine)
IAM role
Select an IAM role to attach to your instance or create a new role if you haven't created any. The role you select replaces any roles that are currently attached to your instance.
instanceRole
Create new IAM role
Cancel
Update IAM role

Once the Attacker-Machine EC2 instance was connected, the Amazon Linux terminal used to run the **aws s3 cp s3://project3-victim-data-17476/sensitive_data.txt** command which exfiltrated the sensitive file from the victims bucket, pulling the data through the VPC network to generate the necessary **VPC Flow Logs** and **CloudTrail** S3 Data Events for the investigation.

```
[ec2-user@Attacker-Machine ~]$ aws s3 cp s3://project3-victim-data-17476/sensitive_data.txt .
download: s3://project3-victim-data-17476/sensitive_data.txt to ./sensitive_data.txt
```

PHASE 3: FORENSIC LOG ANALYSIS

Once the data exfiltration was complete, the logs are actively being delivered to the S3 bucket. To analyze them, two tables in **Amazon Athena** were created. One for the **CloudTrail API logs** (to see *who* accessed the file) and one for the **VPC Flow Logs** (to see *where* the data was sent).

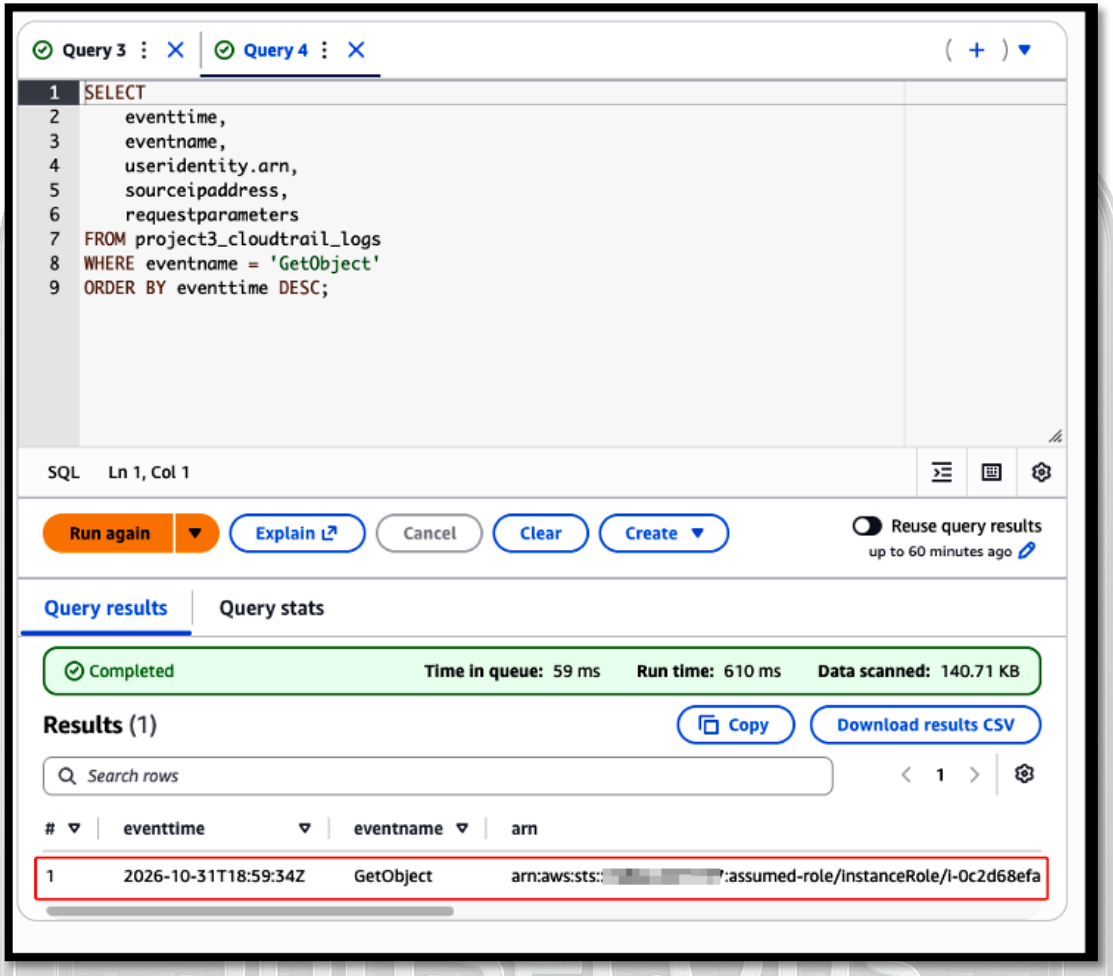
CLOUDTRAIL TABLE

```
CREATE EXTERNAL TABLE project3_cloudtrail_logs (
    eventVersion STRING,
    userIdentity STRUCT<type: STRING, principalId: STRING, arn: STRING, accountId: 
    eventTime STRING,
    eventSource STRING,
    eventName STRING,
    awsRegion STRING,
    sourceIPAddress STRING,
    userAgent STRING,
    errorCode STRING,
    errorMessage STRING,
    requestParameters STRING,
    responseElements STRING,
    additionalEventData STRING,
    requestId STRING,
    eventId STRING,
    readOnly STRING,
    resources ARRAY<STRUCT<arn: STRING, accountId: STRING, type: STRING>>,
    eventType STRING,
    apiVersion STRING,
    recipientAccountId STRING,
    serviceEventDetails STRING,
    sharedEventID STRING,
    vpcEndpointId STRING,
    tlsDetails STRUCT<tlsVersion: STRING, cipherSuite: STRING, clientProvidedHostHe
)
ROW FORMAT SERDE 'com.amazon.emr.hive.serde.CloudTrailSerde'
STORED AS INPUTFORMAT 'com.amazon.emr.cloudtrail.CloudTrailInputFormat'
OUTPUTFORMAT 'org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat'
LOCATION 's3://project3-exil-logs-718/AWSLogs/[redacted]/CloudTrail/';
```

VPC FLOW LOG TABLE

```
CREATE EXTERNAL TABLE vpc_flow_logs (
  version int,
  account_id string,
  interface_id string,
  srcaddr string,
  dstaddr string,
  srcport int,
  dstport int,
  protocol int,
  packets bigint,
  bytes bigint,
  `start` int,
  `end` int,
  action string,
  log_status string
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ' '
LOCATION 's3://project3-exil-logs-718/AWSLogs/[redacted]/vpcflowlogs/'
TBLPROPERTIES ("skip.header.line.count"="1");
```

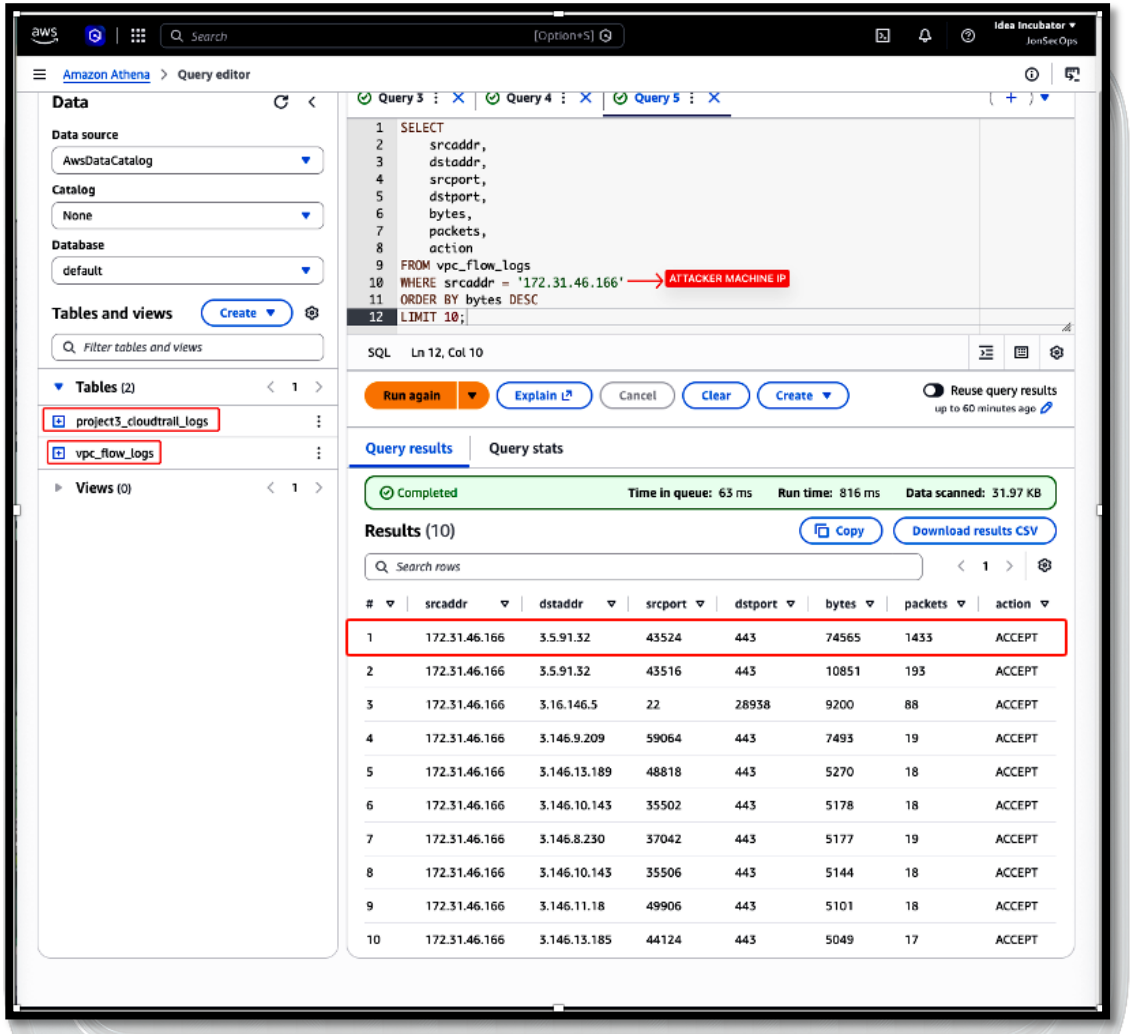
Once both forensic tables were configured, the API level in CloudTrail was analyzed to pinpoint the exact moment the sensitive file was accessed. The following SQL command was used to filter the CloudTrail for file download **GetObject** events.



To show the entire result of the query, Amazon Athena **Download results CSV** utility was used and displayed the source IP address from the CloudTrail event. This successfully identified the exact moment the file was stolen. The results confirm that the IAM role attached to the EC2 instance (instanceRole) performed the GetObject action to download the sensitive data.

eventtime	eventname	arn	sourceipaddress	requestparameters
2026-10-31T18:59:34Z	GetObject	arn:aws:sts::[redacted]	3.137.155.224	{"bucketName":"project3-victim-data-17476","Host":"project3-victim-data-17476.s3.us-east-2.amazonaws.com","key":"sensitive_data.txt"}

Now that the source IP address from that CloudTrail event was obtained, the VPC Flow Logs was queried to correlate the API call with the actual network traffic and measure exactly how much data was exfiltrated. The following SQL command was used.



The SQL query displayed what is needed to prove the exfiltration. Row 1 reveals the largest data transfer (74,565 bytes) originating from the attacker machine's private IP (172.31.46.166) and traveling to an external AWS IP (3.5.91.32) over port 443 (HTTPS). This specific network flow is the underlying transport layer record of the aws s3 cp command pulling the sensitive_data.txt file down to the EC2 instance.

PHASE 4: RECOMMENDATIONS

- **Enforce VPC Endpoint Policies**: Route S3 traffic through a VPC Gateway Endpoint and apply strict resource policies to prevent data from being pulled to unauthorized external IP addresses or untrusted VPCs.

- **Implement Data Perimeter Controls**: Utilize AWS IAM condition keys (such as aws:SourceVpc and aws:SourceIp) to ensure sensitive S3 buckets can only be accessed from approved corporate networks and authorized instances.

- **Enable Automated Threat Detection**: Deploy Amazon GuardDuty to automatically monitor CloudTrail S3 Data Events and VPC Flow Logs for anomalous exfiltration patterns, providing real-time alerts for suspicious EC2 behavior.

7339 86D1 JSO JonSecOps

APPENDIX: PICERL INCIDENT RESPONSE LIFECYCLE MATRIX

IR Phase	Actions Executed	Key Findings & Outcomes
Preparation	Configured an Amazon S3 evidence bucket (project3-exil-logs-718) and enabled AWS CloudTrail (with S3 Data Events) alongside VPC Flow Logs to establish comprehensive data-plane visibility.	Ensured both management-plane API events and network-level transport layer data were actively captured and centrally stored prior to simulation.
Identification	Executed forensic SQL queries via Amazon Athena against the project3_cloudtrail_logs schema to analyze control-plane activity.	Identified the exact moment of the breach by isolating the GetObject API event, revealing the compromised IAM role (Attacker-S3-Role) and the threat actor's private IP address.
Containment	(Simulated) Evaluated network boundary controls; determined that the compromised EC2 instance (Attacker-Machine) must have its IAM role detached and its Security Group isolated to halt further internal lateral movement and data theft.	Highlighted the immediate need to restrict egress traffic and revoke excessive S3 permissions to prevent ongoing unauthorized access.
Eradication	(Simulated) Removed the compromised IAM role from the EC2 instance and deleted the exfiltrated payload (sensitive_data.txt) from the attacker's local directory.	Successfully neutralized the threat actor's ability to pull additional sensitive objects from the target S3 bucket (project3-victim-data-17476).
Recovery	Correlated the identified CloudTrail IP address with the vpc_flow_logs Athena schema to measure network egress via the transport layer.	Successfully quantified the total data loss (74,565 bytes) and mapped the egress routing path (HTTPS/Port 443), determining the full scope of the breach.
Lessons Learned	Analyzed the exfiltration vectors; recognized that standard S3 configurations allow data to be routed out to public internet destinations without restriction.	Established the requirement to enforce VPC Endpoint Policies and IAM Data Perimeter controls to ensure sensitive S3 buckets are strictly isolated.